

Development of A New HPL-AI for Commodity Servers and Supercomputers using AMD GPUs

Hui Liu, Pak Nin Lui

Client Solution Group / Data Center GPU and Accelerated Processing

hui.liu1@amd.com, pak.lui@amd.com

ABSTRACT

HPL-AI [1] is a mixed precision implementation of the HPL benchmark solving a dense system of linear equations that has been used for evaluating performance for machine learning. The HPL-AI has two parts, which are LU factorization and iterative refinement. The LU factorization is performed in lower precisions (single and half precisions) and used as a preconditioner for an iterative refinement solver which brings the solution vector to full precision. In this paper, we will first describe our implementation of HPL-AI benchmark that is vendor-agnostic, and demonstrate the performance achieved including test runs on commodity Gigabyte and Supermicro servers with AMD Instinct MI210, as well as the ORNL Crusher (also known as ORNL Frontier TDS supercomputer) [2], the pre-deployed ORNL Frontier supercomputer and also the recently introduced CSC LUMI supercomputer, which are based on AMD Instinct MI250X GPUs. We also compared performance of the ORNL Frontier TDS and LUMI supercomputers with the Instinct MI250X to the NVIDIA DGX A100 on a single-node basis, and our HPL-AI on MI250X was 2.6 times faster than A100. We also demonstrated the implementation to achieve linear (and sometimes super-linear) scaling for up to 512 nodes on the pre-deployed Frontier and LUMI supercomputers. The novelties of our implementation over the NVIDIA and ORNL implementations are for developing new numerical methods, designing parallel algorithms and new networking types, for reducing the CPU dependency for Iterative Refinement, by taking advantage of the higher memory bandwidth of the AMD Instinct GPU, and by eliminating the need for the expensive data transfers to CPU cores, and for offloading work from CPU, thus reducing CPU cost and providing much higher power efficiency.

Keywords: HPL-AI, GPU, Performance Optimization, MPI (Message Passing Interface), Communication

1 Introduction

The HPL-AI benchmark seeks to highlight the emerging convergence of high-performance computing (HPC) and artificial intelligence (AI) workloads. While traditional HPC focused on simulation runs for modeling phenomena in physics, chemistry, biology, and so on, the mathematical models that drive these computations require, for the most part, 64-bit accuracy. On the other hand, the machine

learning methods that fuel advances in AI achieve desired results at 32-bit and even lower floating-point precision formats. This lesser demand for accuracy fueled a resurgence of interest in new hardware platforms that deliver a mix of unprecedented performance levels and energy savings to achieve the classification and recognition fidelity afforded by higher-accuracy formats.

1.1 Numerical Methods and Precision Used

HPL-AI strives to unite these two realms by delivering a blend of modern algorithms and contemporary hardware while simultaneously connecting to the solver formulation of the decades-old HPL framework of benchmarking the largest supercomputing installations in the world. The solver method of choice is a combination of LU factorization and iterative refinement performed afterwards to bring the solution back to 64-bit accuracy. The innovation of HPL-AI lies in dropping the requirement of 64-bit computation throughout the entire solution process and instead opting for low-precision (likely single and 16-bit) accuracy for LU, and a sophisticated iteration (double) to recover the accuracy lost in factorization, which uses application of the L and U factors to serve as a preconditioner. The combination of these algorithms is demonstrably sufficient for high accuracy and may be implemented in a way that takes advantage of the current and upcoming devices for accelerating AI workloads.

1.2 Prior Work and Collaboration

The HPL-AI from RIKEN-RCCS Fugaku had been No 1 in the world for two years, and it reached 2 EFLOPS, which is averaged to 12.58 TFLOPS per node. RIKEN-RCCS also released part of its code [3]. The Oak Ridge National Laboratories (ORNL) developed its own HPL-AI implementation and achieved No 1 Jun 2022 [4]. Its performance was 6.86 EFLOPS, which is averaged to around 185.4 TFLOPS per MI250X and 92.7 TFLOPS per GCD. NVIDIA improved their HPL-AI, and they claimed their new HPL-AI was 2x faster than previous version [5].

2 Algorithms

HPL-AI is a mixed precision implementation of the HPL benchmark solving a dense system of linear equations. The LU factorization is performed in lower precision and used as a preconditioner for an iterative refinement solver which brings the solution vector to full precision.

Based on the hardware architectures, developers have different choices for implementing HPL-AI. For example, we may put all computations on GPUs, and we may put

some computations on CPUs and some on GPUs. For computation on CPUs, we may utilize OpenMP to accelerate. Different choices impact the final performance of HPL-AI.

2.1 LU Factorization

The LU factorization [6] is standard LU factorization using panels. The right looking method is applied, which overlap the computations and communications. While the standard HPL uses double precision data type for all computations, HPL-AI uses single precision data type for LU factorization, however, when it comes to matrix-matrix multiplication, half precision (FP16) is used. The reason is that the half precision computations have much higher performance than single precision and double precision. For example, the peak half precision (FP16) performance of MI250X is 383 TFLOPS, while single precision and double precision have peak performance of 95.7 TFLOPS and 47.9 TFLOPS. In our HPL-AI implementation, all half precision related matrix-matrix multiplication uses the rocBLAS extension API, `rocblas_gemm_ex`, which supports mixed half precision and single precision computations.

2.2 Iterative Refinement

Iterative refinement (IR) is an iterative method proposed by James H. Wilkinson to improve the accuracy of numerical solutions to systems of linear equations. In each iteration, the algorithm involves 3 steps. The first step is to compute its current residual. The second step is to compute the correction using LU preconditioner. The third step is to add the correction to the solution. The iterative refinement will stop if the tolerance condition is satisfied.

2.3 Computing Iterative Refinement on GPU

There are some implementations choose to put the iterative refinement (IR) part on the CPU, such as original Fugaku code. For our implementation, we opted to port the CPU portion of the iterative refinement to GPUs, and also rewrote some algorithms, such as GPU-version random number generator and GPU-version matrix-vector multiplication algorithm. These two algorithms are not straightforward, which require deep understanding of the math behind them.

In our evaluations, the IR part makes a big difference for small scale benchmarks. For example, if the IR is computed on the CPUs at the small scale, such as single node, the IR part could take up to 45% of the total computing time. For large scale benchmarks, the CPU-version IR may take less than 10% of the total computing time.

If the IR is computed on the GPUs, for small scale benchmarks, the IR takes much less time than CPU-version IR. For example, using 8 MI100 on a single node, the GPU-version IR takes around 7.5% of the total computing time. For large scale benchmarks, the IR part takes less than 2% of the total computing time. It looks we are the only one that puts IR on GPUs.

2.4 Panel Broadcast Algorithms for Communications

During LU factorization, when a panel has been computed, this panel needs to be broadcast to other columns. Also, in the trailing submatrix update, the current panel row needs to be broadcast to each process row. There are 6 main broadcast algorithms to choose: increasing ring, modify increasing ring, increasing 2-ring, modified increasing 2-ring, long bandwidth reducing, and modified long bandwidth reducing.

In our HPL-AI, a few new broadcast algorithms have been designed for very large runs, including increasing 3-ring, modified 3-ring, increasing 4-ring and modified increasing 4-ring.

3 Benchmark Configurations

The HPL-AI has been tested thoroughly on various commodity servers and GPU and network interconnects.

For 1-node cases, across different systems, compared to NVIDIA DGX A100 is shown in

Hostname	System	GPU Type	Network	Total TF/s
CSC LUMI	HPE CRAY EX235A	4*MI250X	XGMI	985.41
ORNL Crusher	HPE CRAY EX235A	4*MI250X	XGMI	876.12
Lockhart	HPE CRAY EX235A	4*MI250X	XGMI	866.74
ubb-r09-13	GBT	4*MI250	XGMI	886.37
mun-gpu05	SMC	8*MI210	XGMI	876.06
mun-gpu03	SMC	8*MI100	XGMI	699.07
DGX A100	NVIDIA DGX A100	8*A100-80GB	NVswitch	763.30

Table 1. Note that performance is measured using default GPU clock speed, e.g. without adjusting the clock speed. For the study of varying clock speeds (e.g. using the “agt_internal” tool), you can review that in detail at the end of the document for the SCLK Speed section.

Hostname	System	GPU Type	Network	Total TF/s
CSC LUMI	HPE CRAY EX235A	4*MI250X	XGMI	985.41
ORNL Crusher	HPE CRAY EX235A	4*MI250X	XGMI	876.12
Lockhart	HPE CRAY EX235A	4*MI250X	XGMI	866.74
ubb-r09-13	GBT	4*MI250	XGMI	886.37
mun-gpu05	SMC	8*MI210	XGMI	876.06
mun-gpu03	SMC	8*MI100	XGMI	699.07
DGX A100	NVIDIA DGX A100	8*A100-80GB	NVswitch	763.30

Table 1: Node Configuration and Performance Achieved

4 Performance

4.1 Performance on Single Node

We compare performance achieved on a single node and compare with what we can run on a NVIDIA DGX-A100 which has 8 NVIDIA A100-80GB.

The HPL-AI software on NVIDIA DGX A100 platform is obtained from the NGC container to conduct the comparison. We use the preset configuration file for DGX-A100. The DGX-A100 contains 80GB A100s [7]. The performance achieved on a single node DGX-A100 equipped with 8 NVIDIA A100-80GB GPUs is 763.3 TF, which is 95.4 TFLOPS per A100.

The performance achieved on a single node of ORNL Crusher node is 876.12 TFLOPS across 4 AMD Instinct MI250X GPUs, which is 219 TFLOPS per GPU and 109.5 TFLOPS per GCD. The performance achieved on a single node of CSC LUMI node is 985 TFLOPS across 4 AMD Instinct MI250X GPUs, which is 246 TFLOPS per GPU and 123 TFLOPS per GCD. Performance boost in the recently collected results on LUMI over the previously collected results on Crusher is likely contributed by the improvements in ROCm software libraries in ROCm 5.3 compared to the older ROCm 5.1 release available on Crusher at the time.

On a per-system level, the performance advantage of the AMD Instinct MI250X on the CSC LUMI platform over the NVIDIA DGX-A100 platform with A100-80GB is by around 29%. On a per-device level, our HPL-AI implementation on MI250X GPU device (each includes 2 GCDs) is over 2.6 times faster than NVIDIA’s HPL-AI on A100 GPU device.

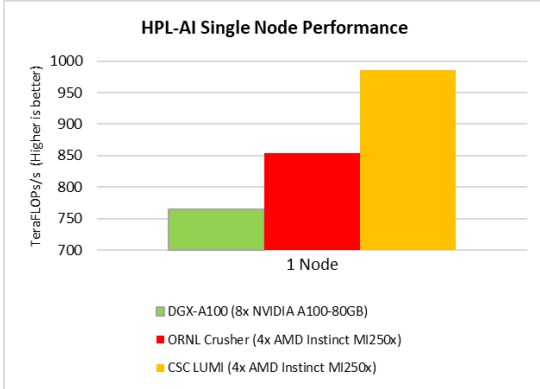


Figure 1: Single Node Performance of ORNL Crusher versus NVIDIA DGX A100 (A100-80GB)

On a Gigabyte G262-Z00 server with 4 MI250, a performance of 886.37 TFLOPS is achieved, which is 221.59 TFLOPS per MI250. It is 2.32 times faster than A100. For mun-gpu05 server with 8 MI210 GPUs, its performance is 876.06 TFLOPS, which 109.51 TFLOPS per GPU. It is 1.15 times faster than A100. Even with MI100, we achieve 699.07 TFLOPS for 8 GPUs and 87.38 TFLOPS per GPU.

The other distinction between AMD implementation and NVIDIA implementation is that AMD implementation does not require a lot of host memory for the CPU. Each of the Crusher node has 512GB while the DGX A100 needs to have 2 TB of system main memory to run the configuration provided.

4.2 Performance at Scale

Scalability is calculated relative to the 1 node with 4x AMD Instinct MI250X GPUs. With the AMD HPL-AI implementation, we can achieve super-linear scaling with certain parameters used.

For reference, the ORNL Crusher performance data for the AMD Instinct MI250X GPUs collected on are shown in the Table 2: Scalability Performance Measured on ORNL Crusher (also known as Frontier TDS Supercomputer) Table 22. The 512-node datapoint is collected using a pre-deployed of the ORNL Frontier, which was configured to lower power consumption and performance. Since we have no access to Frontier, this is the only performance data we have, which has not been tuned for best performance. HPL-AI Configuration details and performance are captured in Table 2.

We recently collected new measurements on the CSC LUMI Supercomputer in Finland in Table 3. The performance of HPL-AI on LUMI can achieve even higher scalability performance than on ORNL Crusher/Frontier. Each GCD of MI250X can achieve peak performance around 135.665 TFLOPs at 64 nodes on LUMI. When using 512 nodes and 4096 GCDs, the HPL-AI achieves 0.522 EFLOPs. Also, the HPL-AI has excellent scalability, when the number of nodes is doubled, its performance is doubled.

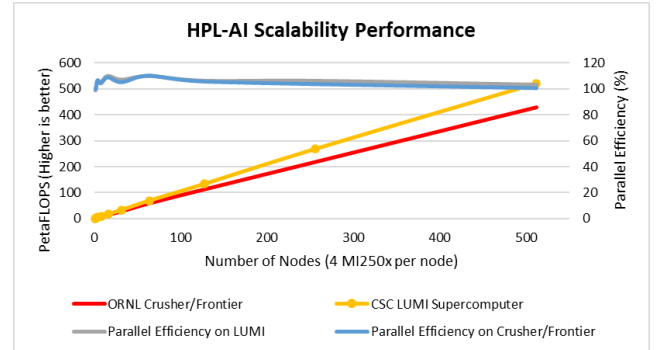


Figure 2: Performance and Parallel Efficiency on ORNL Frontier TDS and Frontier

Nodes	GPUs	N	B	P	Q	Total (s)	LU time (s)	IR time (s)	Total (TF)	Per GCD (TF)
1	8	331776	2560	4	2	29.3623	28.28969	1.07258	876.12	109.515
2	16	479232	2560	4	4	40.1367	38.83906	1.29761	1735.97	108.498125
4	32	663552	2560	8	4	57.399	55.68076	1.71824	3585.4	112.04375
8	64	983040	2560	8	8	85.8399	84.20636	1.63343	7377.92	115.28
16	128	1392640	2560	16	8	119.507	117.3498	2.15744	15067.2	117.7125
32	256	1966080	2560	16	16	166.868	164.0158	2.8524	30362.6	118.6039063
64	512	2785280	2560	32	16	239.9003	235.6356	4.26475	60046.1	117.2775391
128	1024	3276800	2560	32	32	358.5108	354.1929	4.31776	113057.7388	110.407948
512 (Frontier)	4096	7700480	2560	64	64	707.6637	695.911	11.75265	430165.2564	105.0208145

Table 2: Scalability Performance Measured on ORNL Crusher (also known as Frontier TDS Supercomputer)

Nodes	GPUs	N	B	P	Q	Total Time (s)	LU time (s)	IR time (s)	Total Tflops	Per GCD (Tflops)
1	8	348160	2560	4	2	28.55	27.62	0.926	985.411	123.176
2	16	491520	2560	4	4	40.57	39.55	1.019	1950.96	121.935
4	32	696320	2560	8	4	53.47	52.69	0.780	4208.85	131.527
8	64	983040	2560	8	8	76.27	75.41	0.857	8303.56	129.743
16	128	1392640	2560	16	8	103.58	102.37	1.210	17382.7	135.802
32	256	1966080	2560	16	16	149.64	146.91	2.733	33856.3	132.251
64	512	2785280	2560	32	16	207.38	203.59	3.786	69460.5	135.665
128	1024	3932160	2560	32	32	301.78	297.20	4.580	134307	131.16
256	2048	5570560	2560	64	32	429.18	417.48	11.691	268513	131.11
512	4096	7864320	2560	64	64	621.61	606.87	14.740	521643	127.354

Table 3: Scalability Performance Measured on CSC LUMI Supercomputer in Finland

5 Performance Profiling

5.1 Kernel Profiling with rocprof

For kernel level profiling, we use the rocprof profiling utility available in ROCm to help us to give us an overall of the types of Tensile kernels used in HPL-AI. The profile shows approximately 72% of the GEMM calls are spent on half precision GEMM calls, while 27% of the GEMM kernel time spent on single precision GEMM calls.

	A	B	C	D	E
1	Name	Calls	TotalDurat	AverageNs	Percentage
2	Cijk_Aiik_Bijk_HSS_BH_MT128x256x32_MI32x32x	15	4970000884	331333392	25.97287
3	Cijk_Aiik_Bijk_SB_MT128x64x16_MI32x32x2x1_S	2957	3820907246	1292156	19.96779
4	Cijk_Aiik_Bijk_HSS_BH_MT128x96x16_MI32x32x8	14	2898556353	207039739	15.14765
5	Cijk_Aiik_Bijk_HSS_BH_MT128x256x32_MI32x32x	33	2516282184	76250975	13.14991
6	void panel_matgen_off_diag_knl<float>(int, int,	1560	1346466626	863119	7.036539
7	otf_gemv_knl(int, int, double, double const*, dc	120	628961910	5241349	3.28691
8	void getf2_scale_update<float, float*>(int, int, fi	120320	432292493	3592	2.25913
9	Cijk_Aiik_Bijk_HSS_BH_MT256x160x32_MI32x32x	16	345767732	21610483	1.806958
10	void getf2_npvt_check_singularity<float, float*>	120320	302102693	2510	1.578767
11	Cijk_Aiik_Bijk_HSS_BH_MT128x256x32_MI32x32x	5	228302539	45660507	1.193093
12	Cijk_Aiik_Bijk_SB_MT64x64x16_MI32x32x2x1_SN	335	225831004	674122	1.180177

Figure 3: rocprof Profile for HPL-AI (Top Kernels Shown)

5.2 Theoretical Performance Calculation

The AMD Instinct MI250X GPU on peak FP16 performance: 383 TF, peak FP32 matrix performance: 95.7 TF. Note that these figures were based on a MI250X GPU. Based on the percentage of HGEMM and SGEMM, we estimated the theoretical performance of the GEMM for HPL-AI to be around 150 TF. With the performance achieved around 105 TF to 118 TF on average per GCD, we achieved the 70%-79% of the theoretical performance for MI250X GCD.

5.3 MPI profiling

We ran MPI profile to help us understand the MPI communications patterns. From the MPI profile collected using CrayPAT and displayed by the Cray Apprentice2, we can see the topology of communications in point-to-point data flow, we see that most of the (data sent) communications take place to neighboring ranks, as well as some communications to 60 ranks above and below. From the data flow, we may expect similar data communication behavior on a large scale, where data communications between local nodes to the remote nodes on different network switches can occur.

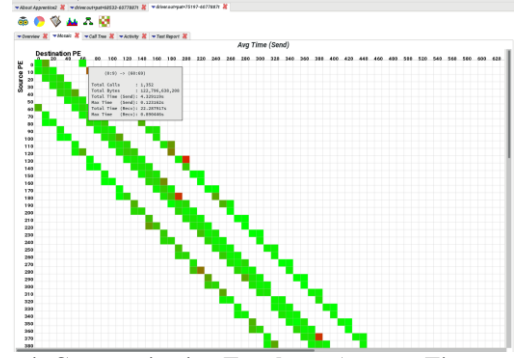


Figure 4: Communication Topology: Average Time sent at 1024 MPI processes (Cray Apprentice2 Profile)

This may affect the design of the network topology (such as switch blocking factor) and switches to handle the large volume of data traffic across remote switches on the network, or the scalability performance on large scale may be affected. More experiments with different MPI communications algorithms can be further explored.

6 Runtime Considerations

6.1 Infinity Fabric (XGMI)

There is a significant performance difference when Infinity Fabric is not used (can be as high as 20%), due to the data transfers occurred in HPL-AI. For example, around 87 TF per MI210 without XGMI vs 105 TF per MI210 with XGMI on single system. Therefore, having Infinity Fabric support is highly recommended.

We ran IPM, which is an open source MPI profiler, the MPI communication account for approximately 28% of the overall runtime on single node. From the profile, we see that majority of the communication takes place in non-blocking communications, such as MPI_Isend/MPI_Irecv/Ibcast and MPI_Test/MPI_Wait.

The distribution of the messages for the non-blocking MPI_Irecv/MPI_Wait are spread across different buffer sizes, mostly in the mid-range messages from 16KB to large messages to over 256MB. However, due to the use of non-blocking communications, the effect of network is not as clear because the network transfer time is overlapped with computation.

6.2 Compute Unit (CU) frequency

Performance of HPL-AI in this report was done with GPU SCLK speed unmodified. It is possible to adjust and set the fan speed to max (as mentioned above), and then SCLK speed to a static speed to achieve higher performance.

For example, on a Gigabyte G262-ZO0-rev-A00 system, performance was seen to increase as much as 10% as the effect of the increase GPU SCLK, from unmodified, to the maximum 1700 MHz, using the AGT internal tool.

```
./agt_internal -i=* -eng=<freq>
```

6.3 Cooling

While testing on various systems, we observed high temperature on the GPU can impact performance of HPL-

AI. Typically, the temperature reported by rocm-smi should show the GPUs/GCDs should be in the temperature of low 30's degrees Celsius. If GPU runs too hot, performance may be throttled. For air-cooled systems, we may want to try to use ipmitool to manually set max fan speed.

6.4 BIOS Setting for NPS / NUMA Per Socket

We observed that by having NUMA Per Socket NPS=4 in the system BIOS, and it is able to get about 4% better performance than having NPS=1. For example, 107 TF per GCD for NPS=1 compared to 110 TF per GCD for NPS=4.

7 Conclusions

This paper presents our work on developing HPL-AI for AMD GPUs. The new parallel iterative refinement algorithm and new network algorithms have been designed. Due to our improved iterative refinement, our HPL-AI on single node is much faster than NVIDIA A100. The HPL-AI is highly scalable, which achieves linear performance boost on LUMI supercomputer. Also, average performance 135.665 TFLOPs per GCD is observed, which is 270 TFLOPs per GPU.

8 References

- [1] University of Tennessee, "HPL-AI Mixed-Precision Benchmark," [Online]. Available: <https://www.hpl-ai.org/>.
- [2] "Crusher Quick-Start Guide," [Online]. Available: https://docs.olcf.ornl.gov/systems/crusher_quick_start_guide.html.
- [3] "An HPL-AI implementation for Fugaku," [Online]. Available: <https://github.com/RIKEN-RCCS/hpl-ai>.
- [4] ORNL, "Frontier supercomputer debuts as world's fastest, breaking exascale barrier," [Online]. Available: <https://www.ornl.gov/news/frontier-supercomputer-debuts-worlds-fastest-breaking-exascale-barrier>.
- [5] NVIDIA, "HPL-AI Now Runs 2x Faster on NVIDIA DGX A100," [Online]. Available: <https://developer.nvidia.com/blog/hpl-ai-now-runs-2x-faster-on-nvidia-dgx-a100/>.
- [6] "HPL Algorithm," [Online]. Available: <https://www.netlib.org/benchmark/hpl/>.
- [7] NVIDIA, "NVIDIA HPC-Benchmarks 21.4," [Online]. Available: <https://catalog.ngc.nvidia.com/orgs/nvidia/containers/hpc-benchmarks>.
- [8] ROCmSoftwarePlatform, "ROCmSoftwarePlatform - HPL-AI," [Online]. Available: <https://github.com/ROCmSoftwarePlatform/hpl-ai>.